# Preparing and running OSMOSE calibrations with calibrar

Ricardo Oliveros-Ramos

2026-04-23

## Table of contents

# 1 Introduction and overview of OSMOSE calibration

This vignette explains how the `osmose` package adapts the generic `calibrar` workflow to a complex, stochastic, disk-based model such as OSMOSE. It focuses on the calibration workspace created by the `osmose_calibration_setup()` function, why that workspace has the structure it does, and how it supports testing, parallel execution, restart capability, and multi-phase optimisation.

The examples below are illustrative. They are intended to explain the workflow and the role of the main files, not to provide a fully runnable end-to-end example inside the `calibrar` package.

> **What is OSMOSE?**
>
> OSMOSE (Object-oriented Simulator of Marine Ecosystems) is a spatially explicit, multi-species, individual-based model that simulates the life history of fish (growth, reproduction, mortality, movement) and their interactions through size-based opportunistic predation. It is a typical example of a complex, stochastic, disk-based model that requires careful calibration against multiple data sources.

The `calibrar` package provides a generic way to fit complex models to data without rewriting the model in R. For simple examples, the full workflow can often be assembled directly in memory. For a complex simulator such as OSMOSE, the same logic usually needs a prepared workspace on disk, with model configuration files, calibration files, data templates, helper scripts, and a stable execution directory.

This vignette explains that prepared workflow. Its purpose is to show how the generic `calibrar` pattern is implemented for OSMOSE through `osmose_calibration_setup()`, and to make the logic of the resulting calibration folder easier to understand for users who need to inspect, edit, test, and run it. Although the example is OSMOSE-specific, the underlying architecture is relevant to other complex models that are controlled from R but executed through model-specific files and wrappers.

## 1.1 What an OSMOSE calibration aims to achieve

At the most general level, an OSMOSE calibration aims to estimate uncertain model parameters so that the OSMOSE core simulator reproduces the main observed features of the system being modelled. In `calibrar` terms, this is still the same basic inverse problem introduced in the other vignettes: define a `run_model()` function, organise the observed data, specify how each target contributes to the objective function, and then optimise the chosen parameters.

For OSMOSE, the uncertain quantities are usually drawn from several parameter families rather than from a single tuning coefficient. These often include parameters related to the initial state of focal populations, fishing-related parameters, early life-stage mortality, and selected trophic or accessibility parameters. Here, "accessibility" refers to parameters controlling how available prey groups are to predators, and "selectivity" refers to how fishing mortality changes with size or age. The calibration therefore combines biological, fishery, and interaction parameters in a constrained and usually multi-phase search.

The model is confronted with several classes of observational targets. In the workflows discussed here, the main targets are typically biomass or abundance indices, catches or landings, and composition data such as catch-at-length. These data-fitting components may be combined with penalty or regularisation terms that do not read any external data file but still contribute to the objective function. This distinction becomes important later when interpreting the `calibration_settings.csv` file.

Throughout this vignette we focus on the default (`type = "simple"`) workflow created by `osmose_calibration_setup()`. This keeps the exposition concrete while still illustrating the full logic of a `calibrar`-based calibration for a complex model. In broad terms, the simple workflow is designed around:

- a selected set of OSMOSE calibration parameters, typically including initial biomass, fishing-related parameters, larval mortality components, and accessibility parameters;
- a selected set of observed targets, typically including biomass indices, catch or landings, and catch-at-length data; and
- a corresponding objective function that combines data-fitting terms and internally generated penalties.

Other OSMOSE calibration templates may also be available depending on the package version and modelling context. Those are better documented in OSMOSE-specific documentation. Here,

the aim is not to catalogue all OSMOSE workflows, but to use one representative workflow to explain how a complex-model calibration is prepared and run through `calibrar`.

# 2 Adapting the generic `calibrar` workflow to OSMOSE: the role of `osmose_calibration_setup()`

## 2.1 The generic `calibrar` pattern

At a conceptual level, the workflow used by `calibrar` is simple. A model-specific function, usually called `run_model()`, receives a set of parameter values, runs the model, and returns the simulated quantities that will be confronted with data. These simulated outputs are then matched to a corresponding set of observed data. A calibration settings table specifies how each model output contributes to the objective function, for example through a log-normal likelihood, a multinomial likelihood, or a penalty term. From these elements, `calibration_objFn()` builds the objective function, and `calibrate()` searches for the parameter values that minimise it.

This separation is one of the main strengths of `calibrar`. The optimiser does not need to know how the model is internally implemented. It only needs a function that can evaluate the model for a given set of parameters and return the outputs required for calibration. In earlier `calibrar` vignettes, this structure is often illustrated with `calibrar_demo()`, which prepares the necessary objects in a compact and pedagogical way. For OSMOSE, the same logic applies, but the implementation needs more infrastructure.

## 2.2 Why complex models need a prepared workflow

For complex models, the generic pattern above is usually not enough on its own. Several practical complications arise.

First, the model may be implemented outside R. OSMOSE is run through the OSMOSE core simulator, so the calibration cannot rely on an in-memory R function alone. Parameters must be written in a format the simulator can read, the simulator must be launched, and the resulting outputs must then be read back into R.

Second, model execution may depend on a set of auxiliary files rather than on a single function call. A calibration therefore needs a stable directory structure in which parameter files, configuration files, output settings, temporary run folders, and simulated outputs all appear in predictable places.

Third, objective-function evaluations can be computationally expensive. A single calibration may require many thousands of model runs, and each of these runs may take long enough that parallel execution and restart capability become essential rather than optional.

Fourth, calibration parameters are not always represented in the same way as model parameters. For optimisation, it is often useful to work on transformed scales, to impose box constraints, and to activate parameters progressively through multiple phases. This introduces an additional calibration layer between the optimiser and the model itself.

For these reasons, a complex-model calibration often needs a prepared workflow rather than a loose collection of scripts. The purpose of the preparation step is to turn the conceptual ingredients expected by `calibrar` into a reproducible and testable calibration workspace.

## 2.3 What the `osmose` package adds

The `osmose` package provides this prepared workflow through `osmose_calibration_setup()`. In practical terms, this function plays for OSMOSE a role analogous to the one played by `calibrar_demo()` in simpler examples, but with two important differences.

First, it is model-specific. It knows how to identify common OSMOSE calibration parameters, how to write OSMOSE parameter files, how to prepare the directory used by the OSMOSE core simulator, and how to generate templates for the observed data expected by the default OSMOSE calibration workflow.

Second, it works on disk rather than only in memory. Instead of returning a small list of objects ready to use in the current R session, it creates a structured calibration directory containing the files needed to test, run, and monitor a calibration. This includes parameter files, calibration settings, helper scripts, and a `master/` directory from which the OSMOSE core simulator can be executed.

Seen from the point of view of `calibrar`, `osmose_calibration_setup()` is therefore not an alternative to the generic workflow. Rather, it is an OSMOSE-specific implementation of that workflow. It translates the generic concepts implemented in `calibrar` into a concrete file-based workflow suited to a complex stochastic ecosystem model.

## 2.4 The role of `osmose_calibration_setup()`

The central role of `osmose_calibration_setup()` is to prepare the calibration workspace before any optimisation is launched. It does not run the calibration itself. Instead, it assembles the pieces that the calibration will later need.

At a conceptual level, the function performs four main tasks.

First, it defines the calibration parameter layer. This includes identifying which OSMOSE parameters belong to the default calibration workflow, creating initial values for them, generating lower and upper bounds, and assigning calibration phases. These are written to the parameter files used later by `calibrar`.

Second, it defines the observation layer. It runs OSMOSE once with the prepared configuration, extracts the simulated outputs used by the workflow, and uses their structure to generate templates for the corresponding observed data. This is important because the observed data are expected to match the simulated outputs in names, dimensions, and aggregation level.

Third, it prepares the execution layer. The function creates the `master/` directory used by the OSMOSE core simulator, writes the calibration-specific OSMOSE configuration files, and copies the helper scripts that link OSMOSE to `calibrar`. These include the default `run_model.R` wrapper and the calibration launcher scripts.

Fourth, it performs technical checks. The setup process is designed to fail early if the model cannot be run with the prepared files or if the expected calibration outputs cannot be produced. This is a major practical advantage, because it allows users to detect technical problems before they start a potentially long calibration.

For OSMOSE users, understanding these roles helps explain why `osmose_calibration_setup()` creates so many files. The function is not merely writing defaults. It is formalising the contract

between the OSMOSE core simulator and the generic optimisation machinery provided by `calibrar`.

### 2.4.1 First setup pass: creating the calibration workspace

A typical first call to `osmose_calibration_setup()` looks like this:

```
calibration_path = osmose_calibration_setup(
  input = config_file,
  osmose = path_to_jar_file,
  type = "simple",
  name = "my_calibration"
)
```

In this vignette we restrict attention to `type = "simple"`, which is the most appropriate entry point for explaining the workflow. The first setup pass should be understood mainly as a workspace-construction step. It creates the calibration directory, prepares the parameter files, writes the helper scripts, generates templates for the expected observed data, and runs the technical checks needed to confirm that the workflow is internally consistent.

This separation is useful in practice. The first pass answers the question "can this calibration workflow run at all?". The second pass, described later, answers the question "do the observed data fit this prepared workflow?". Keeping these two steps apart makes early failures easier to diagnose and keeps the structure of the calibration directory easier to understand.

> **i** Why two passes?
>
> - **First pass** → answers "Can the workflow run at all?" (tests infrastructure)
> - **Second pass** → answers "Do the observed data fit the workflow?"

## 3 Understanding the calibration directory

After the first setup pass, the calibration workflow becomes visible as a structured directory on disk. This directory should not be interpreted as a miscellaneous collection of files. It is the concrete implementation of the workflow contract between `calibrar`, the OSMOSE helper scripts, and the OSMOSE core simulator.

For a simple model example in `calibrar`, the key ingredients of a calibration may exist directly in the current R session as objects such as `observed`, `setup`, `par`, `lower`, `upper`, and `phase`. In the OSMOSE workflow, the same conceptual ingredients are prepared as files and helper objects inside a dedicated calibration directory. This is what allows the workflow to remain stable across repeated runs, different machines, and long HPC jobs.

A typical calibration directory may look like this:

```
calibration/
   .calibrarrc
   .calibrartest
   .calibration.R
   .simulated.rds
   observed.rds
   calibration_MPI.pbs
   calibration_OMP.pbs
   calibration_OMP.slurm
   calibration_sequentiel.pbs
   calibration_sequentiel.slurm
   osmose-calibration_settings.csv
   osmose-parguess.osm
   osmose-parmax.osm
   osmose-parmin.osm
   osmose-parphase.osm
   run_model.R
   master/
       calibration_parameters.osm
       osmose-calibration.osm
```

### 3.1 Main files created by setup

The files created by `osmose_calibration_setup()` can be grouped according to their role in the workflow.

A first group consists of the optimisation control files. The hidden files `.calibration.R`, `.calibrarrc`, and `.calibrartest` form the main bridge to `calibrar`. The file `.calibration.R` contains the launch logic for the calibration itself. The file `.calibrarrc` stores the main optimisation control options, such as the optimisation method, report frequency, stopping criteria, and phase-specific tolerances. The file `.calibrartest` is a lighter control file used for short test runs. Together, these files define how the optimisation is launched and controlled once the workspace has been prepared.

A second group consists of the calibration parameter files. These are the files ending in `-parguess.osm`, `-parmin.osm`, `-parmax.osm`, and `-parphase.osm`. They contain, respectively, the initial parameter values, the lower bounds, the upper bounds, and the calibration phases for the optimisation parameters. These files define the parameter layer used by `calibrar`; they are not simply another copy of the model configuration.

A third group consists of the objective-function input files. The file `osmose-calibration_settings.csv` defines which outputs contribute to the objective function and how they are treated. The file `observed.rds` stores the observed data after they have been read and reorganised into the structure expected by the workflow. The hidden file `.simulated.rds` stores a reference simulated output produced during setup. It is mainly used for compatibility checks, helping verify that observed and simulated data have matching names and dimensions before the calibration is launched.

A fourth group consists of the model-wrapper and launch files. The most important of these is `run_model.R`, which implements the OSMOSE-specific `run_model()` function used by the

objective function. In addition, setup may create scheduler scripts such as `calibration_OMP.pbs`, `calibration_MPI.pbs`, or corresponding Slurm launch scripts, depending on the intended execution environment.

For practical work, it is useful to remember that these files belong to different conceptual layers. The parameter files define what is estimated. The settings file defines how the fit is measured. The observed-data object stores the external information that will be confronted with the model. The wrapper and launch files define how model evaluations are actually run under optimiser control.

### 3.2 The `master/` directory

The `master/` directory is the execution anchor for the OSMOSE core simulator. It contains the calibration-specific OSMOSE configuration used for model runs, including the calibration parameter file written at each evaluation and the main OSMOSE calibration configuration.

This separation between the calibration directory and the `master/` directory is important. The calibration directory is the overall workspace. The `master/` directory is the reference run directory from which the OSMOSE core simulator can be executed. In the language of `calibrar`, this corresponds naturally to the distinction between a `master` directory, which provides the baseline run context, and the temporary run directories that may later be created when evaluating different parameter combinations in parallel, using `master` as a template.

In other words, the calibration workspace is not only a place to store optimisation settings. It also contains an executable model environment. This is one of the main reasons the workflow must be prepared on disk rather than assembled only as in-memory R objects.

### 3.3 The `data_templates/` directory

The `data_templates/` directory belongs to the observation layer of the workflow. It is created from the structure of the simulated outputs produced during setup. Its purpose is to show the user exactly what observed data the current workflow expects.

This directory is especially useful because it makes the expected correspondence between simulated and observed data explicit. Instead of asking the user to guess the file names, frequencies, dimensions, or aggregation levels required by the calibration, setup produces file templates that already match the current OSMOSE configuration and output settings.

These templates should therefore be interpreted as a formal interface between model outputs and observed data. They are not generic CSV examples. They are generated from the actual structure of the current calibration workflow and the exact format and structure needs to be preserved.

## 4 Preparing, connecting, and finalising observed data

The next stage of the workflow concerns the observed-data layer. Once setup has created the expected templates, the user must decide which observed datasets will actually be used for the calibration and provide them in the expected structure.

## 4.1 How templates are generated

The templates created by `osmose_calibration_setup()` are derived from a test simulation run and from the current OSMOSE configuration. Their structure therefore depends on the outputs activated in the model, the output frequency, and the species or fisheries represented in the configuration.

For the `simple` workflow, the default templates typically correspond to a subset of commonly used time-series calibration targets, such as biomass indices, catch or landings, and catch-at-length compositions. The exact list depends on what the model is currently configured to produce and how these outputs are organised by the helper functions.

This is an important conceptual point. In the workflow implemented here, observed data are not defined independently of the model outputs. Instead, the expected observed-data structure is generated from the simulated outputs, and the user is then asked to provide matching datasets. This greatly reduces ambiguity in the construction of the objective function.

## 4.2 Which files the user is expected to fill

The templates in `data_templates/` indicate the files that the current workflow expects the user to populate with observed data. In a typical `simple` workflow, these may include one or more biomass index files, a catch or landings file, and one catch-at-length file per species for which composition data are intended to be used.

The key requirement is not the specific file names themselves, but the consistency between the file structure and the simulated outputs. The observed data should match the expected variable names, time indexing, and dimensions implied by the templates. Changing this structure manually is usually a bad idea, because later compatibility checks rely on the expected layout.

In practice, many users will prefer to copy the `data_templates/` directory to a separate working location before inserting the real data. This is often the safest approach, because it preserves the original templates while allowing the user to organise and edit the actual datasets elsewhere.

## 4.3 When data files should be removed rather than filled

Not every simulated output necessarily has an observational counterpart. In some cases, the workflow may generate a template because the corresponding model output is available, but the user may not have a usable observed dataset for that target.

In this case, the correct action is usually not to invent a placeholder dataset, but to remove the file from the observed-data folder and switch off the corresponding target in `osmose-calibration_settings.csv`. This distinction is important because the presence of a template reflects what the model can produce, not what the user must necessarily include in the calibration.

This is one of the reasons the calibration settings file is so central. The templates expose the possible observation targets, while `osmose-calibration_settings.csv` records which of these targets are actually active in the objective function.

### 4.4 Finalising the workspace with observed data

Once the user has prepared the observed-data files, a second call to `osmose_calibration_setup()` is used to finalise the workspace and connect it to the real data location.

```
calibration_path = osmose_calibration_setup(
  input = config_file,
  osmose = path_to_jar_file,
  type = "simple",
  name = "my_calibration",
  data_path = "real_data"
)
```

The purpose of this second setup pass is different from that of the first one. The first pass mainly constructs and tests the calibration workspace. The second pass links that workspace to the observed data, reads these data into the internal structure expected by the calibration, stores them as `observed.rds`, and reruns the compatibility checks between observed and simulated outputs.

This two-stage logic is particularly useful for complex models. It allows users to separate problems caused by technical workflow issues from problems caused by the observed data themselves. If the first setup pass fails, the issue is usually in the model configuration, the execution environment, or the workspace definition. If the first pass succeeds but the second fails, the issue is more likely to lie in the correspondence between the observed data and the simulated outputs.

This also explains why rerunning setup is not redundant. The second pass is part of the normal workflow. It is the stage at which the prepared workspace becomes a calibration-ready workspace.

## 5 Understanding the `calibration_settings.csv` file

Among all files created during setup, the `calibration_settings.csv` file is one of the most important to inspect before a first calibration. It defines how the simulated outputs returned by `run_model()` are translated into objective-function components.

### 5.1 Why this file is central

At the conceptual level, this file is the specification of the objective-function layer. Each row represents one calibration target or one objective-function component. The file records which simulated variable is concerned, how it should be compared with observed data, whether it is active in the calibration, where the data file is located, and which additional identifiers or options are associated with it.

For simple examples in `calibrar`, the same information may be prepared directly as an in-memory data frame passed to `calibration_setup()`. In the OSMOSE workflow, this information is materialised as a file because it is part of the persistent calibration workspace and is expected to be inspected and edited by the user.

## 5.2 Key columns

A few columns are conceptually central.

The `variable` column identifies the simulated output returned by `run_model()`.

The `type` column specifies the objective-function component used for that variable, for example a log-normal likelihood, a multinomial likelihood, or a penalty-like function.

The `calibrate` column indicates whether the corresponding target is active in the calibration.

The `cv` column records the assumed observation uncertainty or confidence level associated with the target, expressed as a coefficient of variation.

The `use_data` column indicates whether the target actually reads observed data from disk. If it does, the `file` column specifies where those data are stored.

A small illustrative example is shown below.

| variable | type | calibrate | cv | use_data | file | varid |
|---|---|---|---|---|---|---|
| biomass.anchoveta | lnorm2 | TRUE | 0.25 | TRUE | bio_idx.csv | anchoveta |
| yield.anchoveta | lnorm2 | TRUE | 0.05 | TRUE | yield.csv | anchoveta |
| catchatlength.anchoveta | multinom | TRUE | 1.00 | TRUE | cal_anch.csv | |
| biomass.sardina | lnorm2 | TRUE | 0.25 | TRUE | bio_idx.csv | sardina |
| yield.sardina | lnorm2 | TRUE | 0.05 | TRUE | yield.csv | sardina |
| biomass.euphausids | lnorm2 | TRUE | 0.25 | TRUE | bio_idx.csv | euphausids |
| yield.euphausids | lnorm2 | FALSE | 0.05 | TRUE | | euphausids |
| penalty.growth.vonbertalanffy | normp | TRUE | 0.10 | FALSE | | |

This example illustrates several common situations in an OSMOSE calibration. The rows for `anchoveta` show a harvested species contributing three different target types: a biomass index, a yield series, and catch-at-length data. The rows for `sardina` show another harvested species for which only biomass and yield are included here. The `euphausids` rows illustrate a species that is represented in the model but not harvested in the calibration setup: its biomass index is still used, but `yield.euphausids` is excluded from the calibration by setting `calibrate = FALSE`. Finally, `penalty.growth.vonbertalanffy` illustrates a penalty term rather than a direct observational target: it contributes to the objective function, but does not read an external data file, so `use_data = FALSE`.

The precise variable names depend on the prepared workflow, but the logic is always the same: identify the target, choose how it contributes to the objective function, decide whether it is active, and indicate whether observed data are read from disk. The settings table is therefore not just a list of files, but the place where the user defines which simulated outputs are confronted with observations, which are excluded, and which enter the calibration as penalty terms.

### 5.3 `calibrate` versus `use_data`

One of the most important distinctions in this file is the difference between `calibrate` and `use_data`.

> **`calibrate` vs `use_data`**
>
> - `calibrate = FALSE` → the objective function **ignores** this component entirely.
> - `use_data = FALSE` → the component is **active** but does not read observed data (e.g., a penalty term).

Setting `calibrate = FALSE` excludes the corresponding component from the calibration. The objective function will ignore it.

By contrast, `use_data` answers a different question: does this component read observed data from disk? If `use_data = TRUE`, the workflow expects a data file and will compare the simulated output with the corresponding observed values. If `use_data = FALSE`, the observed input is effectively `NULL`, and the objective-function component is expected to operate as a penalty or auxiliary term based only on the simulated values. Setting `use_data = FALSE` when a data component is calibrated and requires data will throw an error.

### 5.4 Typical edits before a first calibration

Before launching a first calibration, users should usually inspect this file carefully.

A common first task is to switch off targets for which no suitable observed dataset is available. If a generated template has no observational counterpart, the corresponding row should usually be deactivated and the unused file removed from the observed-data folder. This will allow to get an error if the variable is accidentaly left active.

A second common task is to review the `cv` values. These values reflect how much confidence is placed in the different datasets and therefore how strongly they influence the overall fit.

A third task is to check that each active target points to the intended file and that the list of active targets is consistent with the contents of the observed-data folder.

## 6 Practical checklist before first launch

Once the workspace has been finalised, a short practical review is usually worthwhile before starting any serious optimisation.

- Inspect `osmose-calibration_settings.csv` and confirm that only the intended targets remain active.
- Inspect `parguess`, `parmin`, `parmax`, and especially `parphase`.
- Run `osmose_calibration_test()` on the prepared directory.

These steps are intended to confirm that the calibration workflow, the data, and the optimisation settings are all coherent. If all the tests implemented in osmose_calibration_test() are passed, you are ready to run the calibration.

**!** Ready to launch the calibration

Passing all the tests in osmose_calibration_test() is the green light to deploy the calibration in production on your local HPC system. At that point, the prepared workflow, the observed data, and the optimisation settings have all passed the main consistency checks expected before launching a full run.

# 7 Understanding the parameter configuration files

The calibration workspace created by `osmose_calibration_setup()` includes a set of files that define the optimisation parameters: initial values, lower and upper bounds, and calibration phases. In the OSMOSE workflow these files are written as `.osm` configuration files rather than as plain vectors or data frames, because they must remain consistent with the parameter conventions used by the model and by the helper scripts.

Conceptually, these files define the parameter layer of the calibration. They specify which quantities are estimated, how they enter the optimisation, and how they are activated over successive phases.

## 7.1 Initial values: `parguess`

The `parguess` file provides the starting values used by the optimisation algorithm. These are values to start the optimisation, not necessarily a direct copy of the values found in the original model configuration.

## 7.2 Bounds: `parmin` and `parmax`

The `parmin` and `parmax` files define the lower and upper bounds used for box-constrained optimisation. For complex models, these bounds are often part of the regularisation strategy and should be checked as carefully as the initial values.

## 7.3 Phases: `parphase`

The `parphase` file defines the phase at which each parameter becomes active in the optimisation. This is one of the most important files to inspect before launching a long calibration.

A small illustrative example makes the distinction clearer:

```
# parguess.osm
population.initialization.log10.biomass.sp0 = 5.20
fisheries.log.rate.base.fsh1 = -1.35
osmose.user.selectivity.delta75.fsh1 = 3.00

# parphase.osm
population.initialization.log10.biomass.sp0 = 1
fisheries.log.rate.base.fsh1 = 2
osmose.user.selectivity.delta75.fsh1 = 5
```

In this example, the three parameters are all present in the calibration, but they do not become active at the same time. The biomass parameter is estimated from phase 1, the fishery-rate parameter from phase 2, and the selectivity helper parameter only from phase 5.

Strategies for defining calibration phases in OSMOSE are described in Oliveros-Ramos et al. (2017), which presents the use of a sequential estimation strategy for a complex OSMOSE application fitted to multiple time series. That paper provides a practical example of how parameters can be activated progressively across phases, rather than estimated all at once, in order to stabilise the optimisation and improve the behaviour of the calibration. Readers interested in the rationale behind phase design, and in concrete examples of how different parameter groups can be introduced step by step, are encouraged to consult that study.

### 7.4 OSMOSE-specific helper logic behind these files

The purpose of `osmose_calibration_setup()` is not only to write these files, but also to translate the OSMOSE configuration into a calibration representation suited to `calibrar`. In the current `simple` workflow, this may involve quantities such as initial biomass, fishing mortality-related parameters, larval mortality components, accessibility parameters, and selectivity terms. Other calibration workflows are currently implemented in `osmose_calibration_setup()` and are described in the OSMOSE documentation, but they implement the same workflow described here.

This translation layer is one of the main reasons the OSMOSE workflow deserves its own helper infrastructure. Without it, users would need to assemble the parameter vector, bounds, and phase structure manually, and also ensure that they remain consistent with the corresponding model-scale parameters.

## 8 The role of `run_model.R`

A central concept in `calibrar` is the `run_model()` function. This function is the bridge between the optimiser and the model. It receives a set of parameter values, runs the model, and returns the simulated outputs to be confronted with observed data.

In the OSMOSE workflow, this bridge is implemented through the `run_model.R` file copied into the calibration directory.

### 8.1 What `run_model()` does in this workflow

In conceptual terms, the OSMOSE `run_model()` wrapper is the model-side bridge between `calibrar` and the OSMOSE core simulator. It receives one candidate calibration parameter set from the optimiser, translates that proposal into a runnable OSMOSE configuration, executes the simulation, and returns the calibration outputs in the structure expected by the objective function. A more detailed walkthrough of this implementation is provided in the appendix describing `run_model.R`.

The function is called through a signature of the form:

```
run_model(par, conf, osmose, is_a_test=FALSE, version="4.3.3",
                      options="-Xmx3g -Xms1g", ...)
```

Here, `par` is the candidate calibration parameter set proposed by the optimiser, `conf` is the prepared OSMOSE calibration configuration, and `osmose` is the path to the OSMOSE executable. The remaining arguments control how the wrapper is run, for example in test mode or under a specific OSMOSE version and Java configuration.

The logic of the wrapper can be summarised in four main steps.

First, it receives the calibration parameters proposed by the optimiser.

Second, it reconstructs the model-level OSMOSE parameters required for the simulation. This may include back-transforming optimisation parameters, expanding grouped parameter representations into species-level values, rebuilding time-varying larval mortality series, reconstructing fishery selectivity terms, and writing the resulting parameter set into the `calibration_parameters.osm` file read by the OSMOSE core simulator.

Third, it launches the OSMOSE simulation in the prepared run directory.

Fourth, it reads and post-processes the simulator outputs so that the result is a named list of calibration variables with the structure expected by the objective function, including both direct observational targets and any penalty-related components returned by the workflow.

A minimal schematic representation looks like this:

```
run_model = function(par, conf, osmose, ...) {
  ## 1. reconstruct simulator-level parameters from calibration parameters
  sim_par = reconstruct_osmose_parameters(par, conf)

  ## 2. write the runnable OSMOSE parameter file
  write_osmose(sim_par, file = "calibration_parameters.osm")

  ## 3. run the OSMOSE core simulator
  run_osmose(input = "osmose-calibration.osm", osmose = osmose, ...)

  ## 4. read the simulator outputs
  output = read_osmose(path = "output", ...)

  ## 5. reduce raw outputs to calibration variables
  cal_output = osmose_calibration_outputs(output)

  ## 6. return the named list used by the objective function
  return(cal_output)
}
```

The real OSMOSE wrapper includes additional steps, such as reconstruction of grouped parameters, rebuilding of larval mortality and selectivity terms, retry logic, and the addition of penalty-related outputs; these are described in the appendix on `run_model.R`.

The exact variable names and contents of `cal_output` depend on the prepared workflow, but the important point is that `run_model()` returns a named list whose structure matches the rows defined in `osmose-calibration_settings.csv`. This is what allows `calibration_objFn()` to compare simulated and observed quantities in a standard way, even though the simulator itself is external to R.

### 8.2 When users may need to edit it

In most cases, the default `run_model.R` file should be sufficient. The helper workflow is designed so that the standard script already performs the parameter reconstruction, simulator launch, and output extraction needed by the pre-packed OSMOSE calibration workflow.

When modifications are needed, they should normally be limited to the explicit user-edit block left in the script. Typical reasons include adding extra derived outputs, modifying how a simulated quantity is aggregated before comparison with observed data, or introducing an application-specific parameter transformation not covered by the default helpers.

For readers who want a fuller explanation of how the wrapper is structured internally, including parameter reconstruction, writing of `calibration_parameters.osm`, retry logic, and the construction of the returned output object, see the dedicated appendix on `run_model.R`.

### 8.3 What is OSMOSE-specific here

The generic pattern is the same for any complex model linked to `calibrar`: receive parameters, run the model, and return a structured list of outputs. What is specific to OSMOSE is how this is achieved. The script must write parameters in OSMOSE format, launch the OSMOSE core simulator in the correct workspace, and interpret OSMOSE outputs into calibration variables such as biomass indices, catch-related quantities, composition data, and penalty terms.

## 9 Running pre-calibration tests

Before launching a full calibration, the prepared workspace should be tested formally. In the OSMOSE workflow, this is done with `osmose_calibration_test()`.

```
osmose_calibration_test(path = calibration_path)
```

These tests should be treated as part of the normal workflow rather than as optional convenience checks.

### 9.1 Testing the prepared workspace

At a practical level, the test function uses the prepared calibration directory, the saved simulated outputs, the observed data, the calibration settings, and the helper scripts to verify that the different layers of the workflow are compatible.

### 9.2 What is being tested

The first type of check concerns compatibility between simulated and observed outputs. The workflow verifies that the variables expected by the calibration have matching names and dimensions, taking into account whether a given objective-function component actually reads observed data.

The second type of check concerns construction of the objective function itself. A calibration-ready workspace should be able to create the objective function from the current setup, observed data, and model wrapper without error.

The third type of check concerns execution mode. The helper workflow can test the calibration path in sequential mode and, if requested, in parallel mode as well.

> **!** Troubleshooting common early failures
>
> The most common early problem is a mismatch between the structure of the simulated outputs and the structure of the observed data. In practice, this usually means that an active target in `osmose-calibration_settings.csv` points to a file whose dimensions, names, or aggregation level do not match what `run_model()` returns.
> When this happens, the most useful checks are: inspect `osmose-calibration_settings.csv`, compare `observed.rds` with `.simulated.rds`, and confirm that the active data files follow the template structure.

# 10 Running calibrations on HPC systems

For many complex models, calibration quickly becomes too demanding for routine execution in a single local R session. This is especially true when the model is stochastic, runs slowly, writes inputs and outputs to disk, or requires many objective-function evaluations. OSMOSE is exactly the kind of model for which HPC execution is often the normal mode rather than a special case.

## 10.1 Why HPC matters for complex model calibration

A calibration does not run the model once. It runs the model repeatedly, often hundreds or thousands of times, under different parameter combinations. When each model evaluation requires writing parameter files, launching an external simulator, reading outputs back into R, and sometimes using internal model parallelisation, the total computational cost becomes substantial.

This is where the design of `calibrar` becomes particularly useful. The optimisation remains generic, but the execution environment can be configured outside the objective function. In practice, this means that the same conceptual workflow can be used in a local multicore session or on an HPC cluster managed by a scheduler.

For readers who want more implementation detail, the appendices provide two complementary views of this process. One appendix explains how the hidden `.calibration.R` launcher reconstructs the prepared calibration problem and turns it into a final call to `calibrate()`. A second appendix shows example PBS and Slurm submission scripts that wrap this launcher for two common scheduler environments.

## 10.2 Launching the calibration script

At the simplest level, the prepared calibration workspace can be launched by executing the calibration script directly:

```
Rscript .calibration.R >>& calibration.log
```

This command is mainly useful to illustrate the logic of the workflow. The real work is done by the hidden calibration script, which reads the prepared files, reconstructs the calibration objects, builds the objective function, configures execution control, and finally launches `calibrate()`. Readers interested in that implementation logic can refer to the appendix describing `.calibration.R`.

## 10.3 Using scheduler scripts

In practice, long OSMOSE calibrations will often be launched through scheduler scripts rather than by calling `.calibration.R` manually.

```
# PBS
qsub calibration_OMP.pbs

# Slurm
sbatch calibration_OMP.slurm
```

These scripts are part of the helper infrastructure copied by `osmose_calibration_setup()`. They provide lightweight scheduler wrappers around the same calibration launcher. Their role is to request resources from the scheduler, move to the calibration directory, set the execution environment, and call `.calibration.R` with the appropriate options.

The exact directives are system-specific, but the conceptual pattern is general. The calibration directory already contains the optimiser-facing files and model-wrapper logic; the scheduler script only needs to provide the platform-specific submission layer. Readers who want concrete examples can refer to the appendix showing PBS and Slurm launch scripts.

## 10.4 Parallelisation and restart capability

One of the strengths of the `calibrar` approach is that parallel execution does not need to be hard-coded into the model itself. Instead, the optimisation workflow can evaluate parameter combinations in parallel while keeping the model-side interface based on `run_model()`. For disk-based models, this requires careful workspace organisation, because concurrent evaluations must run in separate directories while still having access to the baseline model files. The OSMOSE helper workflow addresses this by preparing a workspace that is already structured for repeated and potentially parallel runs.

Restart capability is equally important, but it only becomes active when the calibration control file requests restart output. In practice, this means that the `.calibrarrc` file must define a restart frequency through options such as the reporting interval, so that intermediate optimisation states are written to disk during the run. Once this is configured, the launch script standardises the restart-file basename and passes it to `calibrate()` through the `control` argument. From that point onward, the optimisation can save intermediate restart objects while it is running.

This matters because long calibrations may be interrupted by wall-time limits, queue policies, or technical failures. If restart saving has been activated in the control settings, a partially completed run can be resumed from the latest saved state rather than starting again from the

initial parameter values. In other words, restart capability is not automatic merely because the workflow uses `calibrar`; it becomes operational because the prepared OSMOSE launcher combines a structured workspace, an explicit restart-file name, and control settings that instruct the optimiser to save intermediate states.

For OSMOSE users, this makes long calibrations much more practical under real HPC constraints. For non-OSMOSE users, it highlights a more general lesson: restart behaviour should be treated as part of the execution design of a complex-model calibration workflow, not as an optional afterthought.

# 11 Controlling, monitoring, and resuming the calibration

## 11.1 Optimisation control with `.calibrarrc`

The `.calibrarrc` file should be understood as the optimisation control layer of the workflow. Whereas `osmose-calibration_settings.csv` defines the objective-function components and the parameter files define the search space, `.calibrarrc` defines how the optimisation itself is run.

### 11.1.1 Main control elements

The exact contents depend on the current helper script, but conceptually this file controls the optimisation method, stopping criteria, report frequency, and phase-specific relative tolerances.

A schematic example might look like this:

```
method = "AHR-ES"
maxit = 200
REPORT = 10
reltol = 0.01, 0.005, 0.002, 0.001
```

The meaning of these settings is straightforward: they define the optimisation method, how long each phase may run, how often progress is reported and saved, including for restart purposes, and how tightly each phase is expected to converge.

Changing the optimisation method is usually done here. For example, moving from a stochastic global search to a local derivative-based method is not a change to `run_model()` or to the data layer; it is a change to the optimisation control layer.

### 11.1.2 Why this file matters for a first calibration

For a first calibration, the default values provided in `.calibrarrc` are usually reasonable as an initial starting point. They allow the prepared workflow to be launched without requiring the user to design an optimisation strategy from scratch.

However, users should not assume that these defaults are optimal. In many applications, the calibration can be made more efficient or more robust by adjusting the control settings once the basic workflow has been validated. This may involve changing the optimisation method, modifying

the stopping criteria, or refining the phase-specific tolerances to better suit the behaviour of the model and the scale of the problem.

This is also why `.calibrarrc` belongs in the vignette. It is not an incidental helper file. It is the most visible and editable representation of the optimisation strategy adopted by the prepared workflow.

## 11.2 Monitoring, restarting, and checking results

Once a calibration has been launched, the workflow becomes mainly operational. Even so, it is useful to keep a clear conceptual picture of what should be monitored and how results are re-entered into the analysis workflow.

### 11.2.1 Monitoring the run

In practice, the first level of monitoring is usually the log output and the presence of intermediate restart files. These indicate whether the optimisation is progressing, whether phases are advancing, and whether the model is running stably under the current configuration.

For long jobs on HPC systems, routine monitoring is part of normal calibration management. It is often more useful to check whether the workflow is behaving coherently than to focus immediately on the numerical value of the objective function alone.

### 11.2.2 Reading calibration outputs

Once a run has produced results, the OSMOSE helper workflow provides a way to read them back into an object suited to inspection.

```
calibration = osmose_calibration_check(input=config_file, name="my_calibration")
```

The `osmose_calibration_check()` function returns an object gathering the calibration results, including the best parameter values recovered from the run, the observed data and calibration settings associated with the workspace, and a set of OSMOSE outputs generated from the updated model run.

This step closes the loop between the prepared calibration workspace and the post-run interpretation of results. It allows the user to inspect the best parameter values together with the observed data, settings, and selected outputs associated with the run.

### 11.2.3 What to inspect first

The first things to inspect are usually the active targets, the parameter traces or phase structure, and the best parameter values returned by the calibration. These help answer the most immediate questions: which parts of the objective function were actually active, which parameters were estimated, and whether the resulting solution is plausible.

It is then useful to check whether the fitted outputs are coherent with the intended data sources and aggregation levels. A numerically successful optimisation is not necessarily a scientifically

coherent calibration if the wrong targets were activated or if the data-model correspondence was poorly defined.

# 12 What transfers to other complex models

Although this vignette is centred on OSMOSE, much of the workflow is generic to the way `calibrar` can be used with complex black-box models.

## 12.1 Elements that are generic to `calibrar`

Several elements of the workflow transfer almost directly to any other complex model linked to `calibrar`. In all cases, a model-specific `run_model()` function is needed to receive candidate parameter values, execute the simulator, and return the outputs that will be confronted with data. The observed data must then be organised so that their structure matches the simulated outputs returned by this wrapper. A calibration settings table is also required to define how each output contributes to the objective function. In addition, the optimisation itself needs initial parameter values, lower and upper bounds, and, when a sequential strategy is used, calibration phases. Finally, for long-running problems, practical use often depends on the availability of parallel and restart infrastructure, so that repeated model evaluations can be managed efficiently and interrupted runs can be resumed.

## 12.2 Elements that are specific to OSMOSE

This vignette has focused on the conceptual logic of using `calibrar` with OSMOSE through the prepared helper workflow implemented in the `osmose` package. Its main purpose is to explain why the workspace has the structure it does and how that structure supports testing, optimisation, parallel execution, and restart capability.

What is specific to OSMOSE is the way these generic ingredients are implemented. The calibration-parameter structure depends on OSMOSE concepts and configuration conventions. The helper scripts copied by `osmose_calibration_setup()` are written specifically for the OSMOSE workflow. The expected directory layout and file names follow the logic of the OSMOSE core simulator and its outputs. Likewise, the post-processing that transforms model outputs into calibration variables is tailored to OSMOSE outputs and to the helper choices implemented in the osmose package.

This distinction is useful because it shows how to think about adaptation to other models. The generic `calibrar` pattern remains the same, but the model-specific preparation layer must be redesigned for each modelling framework.

# 13 Appendix: Running a calibration using the `.calibration.R` script.

The script `.calibration.R` is the executable bridge between the prepared calibration directory and the final call to `calibrate()`. In the main text, the calibration workflow is described conceptually in terms of parameter files, observed data, optimisation control, and the `run_model()` wrapper. This appendix explains how those pieces are assembled in practice by the launch script.

This appendix has a double purpose. For OSMOSE users, it provides a more detailed reference for understanding what happens when the calibration is launched from the prepared workspace. For modellers working with other complex simulators, it also illustrates a general implementation pattern: a scheduler-aware R script that reconstructs a calibration problem from files, creates an objective function, configures parallel execution, and then delegates the optimisation itself to `calibrate()`.

At a high level, the script performs the following operations: it reads command-line arguments and scheduler information; it loads the calibration control file and prepares a run directory; it reconstructs the calibration inputs from the prepared workspace; it builds the objective function with `calibration_objFn()`; it configures parallel execution and restart behaviour; and it finally calls `calibrate()` with the prepared objects. In other words, the script translates a calibration directory on disk into the standard `calibrar` call pattern:

```
calibrate(
  par = ...,
  fn = ...,
  lower = ...,
  upper = ...,
  phases = ...,
  control = ...,
  parallel = ...
)
```

The apparent simplicity of that final call hides most of the technical work needed by a complex, file-based simulator such as OSMOSE.

The script begins by establishing the runtime context. It reads command-line arguments, loads the required packages, and detects the execution environment.

```
if(!exists(".args", mode = "character")) .args = commandArgs(trailingOnly=TRUE)

library("calibrar")
library("osmose")

nodefile = Sys.getenv("PBS_NODEFILE")
nodes = if(nodefile=="") "localhost" else readLines(nodefile)

control_file   = .get_command_argument(.args, "calibration.control", default=".calibrarrc")
is_a_test      = .get_command_argument(.args, "test")
run_model_file = .get_command_argument(.args, "run_model", default="run_model.R")
ncores         = .get_command_argument(.args, "ncores", default=length(nodes))
OMP            = .get_command_argument(.args, "omp")
nht            = .get_command_argument(.args, "nht")
replicates     = .get_command_argument(.args, "replicates", default=1)
```

This first block makes the script usable both interactively and under a scheduler. If the script is launched through a PBS job, the node file is used to detect the available nodes. If not, it falls back to a local default. It also translates external execution options into explicit R objects. Rather than hard-coding execution mode, the script makes parallel mode, test mode, run-model file, and

calibration control file configurable from the command line. This is good practice for complex model workflows because the same prepared workspace may need to be launched in different ways. It is also worth noting that scheduler-level information and optimiser-level information are still separate at this point. The number of cores requested from the scheduler, the use of OMP or MPI, and the use of hyper-threading are not yet the same thing as the number of workers eventually passed to `calibrar`. That translation happens later, but the raw information is collected here. For non-OSMOSE users, this is a transferable pattern: parse external arguments early, detect the execution environment, and avoid burying run-specific choices inside the source code.

The next block loads the calibration control file, identifies the executable resources needed by the model, and prepares the run directory.

```
control  = .read_configuration(control_file)
osmose   = ".osmose.jar"
obs_file = "observed.rds"

root = basename(getwd())
run_path = sprintf("../.run_%s", root)
if(!dir.exists(run_path)) dir.create(run_path)
if(!file.exists(osmose)) stop("OSMOSE executable '.osmose.jar' not found.")

.cancopy = file.copy(from=osmose, to=file.path(run_path, osmose), overwrite = TRUE)
if(isFALSE(.cancopy)) stop("Cannot copy OSMOSE executable.")
```

This block is important because `calibrar` does not need a run directory by itself. A simple in-memory model can often be calibrated with no file operations at all. The run directory exists because OSMOSE is a file-based simulator: each model evaluation needs input files on disk, an executable jar, and a place to write outputs. The `.calibrarrc` file is read here because it provides the main optimisation settings before the calibration objects are assembled. In a generic `calibrar` workflow, these settings could have been created directly as an R list. In the OSMOSE workflow, they are stored on disk so that the calibration can be launched consistently from scripts, scheduler jobs, or relaunches. The copy of `.osmose.jar` into the run directory is also part of that logic. The script is not only building an optimisation problem in R; it is preparing an executable model environment for repeated simulation calls. This separation between the calibration workspace and the actual run directory is essential whenever a model writes files during execution. For other models, the exact details will differ, but the pattern is the same: if the simulator depends on external binaries, configuration files, or fixed input resources, the launch script should standardise those resources before the optimiser starts evaluating parameter sets.

The script then rebuilds the calibration problem in R from the files that were prepared earlier by `osmose_calibration_setup()`.

```
conf = read_osmose(input = "master/osmose-calibration.osm")
model = get_par(conf, "output.file.prefix")
osm_ncpu = get_par(conf, "simulation.ncpu")

source(run_model_file, local=TRUE)
if(!exists("run_model", mode="function"))
```

```
  stop(sprintf("We couldn't find the 'run_model' function in '%s'.", run_model_file))

setup = calibration_setup(file = sprintf("osmose-%s-calibration_settings.csv", model))

par_guess = read_osmose(input=sprintf("osmose-%s-parguess.osm", model))
par_min   = read_osmose(input=sprintf("osmose-%s-parmin.osm", model))
par_max   = read_osmose(input=sprintf("osmose-%s-parmax.osm", model))
par_phase = read_osmose(input=sprintf("osmose-%s-parphase.osm", model))

observed = readRDS(obs_file)
```

This is one of the most important parts of the script because it makes clear that the launch step is not constructing a calibration from scratch. All the core ingredients already exist in the prepared workspace: `run_model.R` provides the model-specific wrapper; `calibration_settings.csv` defines the objective-function components; `parguess`, `parmin`, `parmax`, and `parphase` define the optimisation parameter layer; `observed.rds` provides the observed data in the structure expected by the workflow; and `master/osmose-calibration.osm` provides the baseline OSMOSE calibration configuration. In a simpler `calibrar` example, these objects may be created directly in the global environment. Here they are reconstructed from files because the workflow must survive across sessions, machines, and scheduler jobs. The role of `source(run_model_file, local = TRUE)` is especially important. `calibrar` does not need to know anything about OSMOSE internally. It only needs a function called `run_model()` that accepts a parameter object and returns a named list of simulated outputs. That is the central abstraction of the black-box workflow. For non-OSMOSE users, this is probably the most transferable part of the script: a similar launch script for another model would also need to reconstruct one model-specific wrapper function, one objective-function settings table, one observed-data object, one set of starting values, bounds, and phases, and one baseline simulation context.

Once the calibration objects have been reconstructed, the script turns them into the scalar objective function that the optimiser can minimise.

```
osmose = file.path("..", osmose)

objfn = calibration_objFn(
  model = run_model,
  setup = setup,
  observed = observed,
  conf = conf,
  osmose = osmose,
  is_a_test = is_a_test
)
```

This block is the core of the `calibrar` abstraction. At this point, the launch script already has a wrapper able to run the simulator, the observed data, and the calibration settings that describe how each output should be compared with its observed counterpart. The call to `calibration_objFn()` combines these pieces into one function, `objfn`, which behaves like any other objective function accepted by `calibrate()`. In practical terms, every time the optimiser proposes a new parameter set, `objfn` will call `run_model()` with those parameters, obtain a structured list of simulated outputs, compare those outputs with the observed data using the

rules defined in `setup`, and aggregate the partial contributions into a single scalar value. This is precisely what allows a complex file-based simulator to be treated as a black-box optimisation problem. In the OSMOSE case, the script also passes `conf`, `osmose`, and `is_a_test` as additional arguments needed by the wrapper. That is model-specific. The generic pattern is that any extra information required by the wrapper function can be passed at objective-function construction time. For modellers of other complex systems, this is where the main design decision appears. The launch script should not try to implement the likelihood directly. Instead, it should expose a model wrapper and a settings table, and let `calibration_objFn()` assemble the comparison logic in a standard way whenever possible.

Before launching the optimisation, the script adjusts some control values so that they are consistent with the prepared workspace.

```
if(!is.null(control$master)) {
  if(control$master!="master") {
    stop(...)
  }
}

if(!is.null(control$run)) {
  if(control$run!=run_path) {
    warning(...)
  }
}

restart_file = if(is_a_test) "osmose-test" else sprintf("osmose-%s", model)
control$master = "master"
control$run = run_path
control$restart.file = restart_file
```

This block is easy to overlook, but it is one of the reasons the script is robust. The control file may contain values for `master` or `run`, but the OSMOSE helper workflow already imposes a strict directory contract. Rather than allowing those values to drift, the script checks them and then rewrites them explicitly. In effect, it says that this workflow always uses the local `master/` directory as the reference run context, and always uses the derived `.run_*` directory as the run workspace. The restart file name is also standardised here. This is important because long calibrations often need to be resumed after interruption. In a complex file-based model, restart behaviour is not a cosmetic convenience; it is part of the normal workflow. For non-OSMOSE users, the general lesson is that the launch script should be the place where generic optimiser controls are reconciled with model-specific workflow assumptions. If a model requires a fixed run layout, fixed auxiliary resources, or a standard naming scheme for restarts, this is the right layer to enforce it.

The script then translates scheduler cores and OSMOSE internal parallelism into the number of workers that `calibrar` should actually use.

```
eff_cores = floor((ncores - 1)/nht/osm_ncpu)
control$ncores = eff_cores
```

This line looks small, but it captures a real implementation issue in complex-model calibration. There are at least three distinct notions of parallelism in this workflow: the total number of scheduler cores available to the job, the number of threads or processes used internally by one OSMOSE simulation, and the number of parameter combinations that `calibrar` can evaluate concurrently. If those levels are confused, the calibration can oversubscribe the machine and perform poorly, or even fail. The script therefore computes an effective worker count by taking into account hyper-threading and the number of cores used internally by each OSMOSE run. For example, if a job has access to 64 cores and each OSMOSE simulation uses 4 cores internally, then `calibrar` can evaluate at most 16 parameter combinations simultaneously. This type of translation is not specific to OSMOSE. Any model with internal parallelism, expensive external binaries, or multiple nested levels of computation will require a similar reconciliation between scheduler resources and optimiser-level parallelism.

Once the effective worker count is known, the script builds the parallel backend used by `calibrar`.

```
if(isTRUE(parallel)) {

  if(!isTRUE(MPI)) {
    if(!require("doParallel")) stop("Package 'doParallel' not found")
    cl = makeCluster(control$ncores)
    registerDoParallel(cl)
  } else {
    if(!require("doSNOW")) stop("Package 'doSNOW' not found")
    cl = makeCluster()
    registerDoSNOW(cl)
  }

  e = new.env()
  e$conf      = conf
  e$is_a_test = is_a_test
  e$osmose    = osmose

  clusterExport(cl, c("conf", "is_a_test", "osmose"), envir = e)
  clusterEvalQ(cl, library("calibrar"))
  clusterEvalQ(cl, library("osmose"))
}
```

This block makes the calibration executable in parallel. The key point is that parallelism is configured outside the objective function itself. This is consistent with the general design of `calibrar`: the optimiser can use a registered parallel backend, while the model-specific wrapper remains an ordinary R function. The script distinguishes between two execution modes: a local or shared-memory parallel mode using `doParallel`, and an MPI-oriented mode using `doSNOW`. That distinction is OSMOSE-specific only in practical terms. The transferable principle is that the launch script should take responsibility for configuring the backend appropriate to the platform, rather than embedding platform assumptions inside the objective function. The exports to the cluster are also important. Worker processes need access to the extra objects required by the wrapper and objective function. The launch script therefore exports `conf`, `is_a_test`, and `osmose`, and ensures that the required packages are loaded on each worker. For another model,

the exported objects would differ, but the logic would be the same: create the cluster, register it, export the objects needed by the wrapper, and make sure workers can evaluate the objective function independently.

After all the preparation steps, the script finally delegates the optimisation itself to `calibrar`.

```
opt = try(calibrate(
  par = par_guess,
  fn = objfn,
  method = method,
  lower = par_min,
  upper = par_max,
  phases = par_phase,
  replicates = replicates,
  control = control,
  parallel = parallel
))
```

This is the endpoint of the script and, conceptually, the point toward which every previous block was working. By the time `calibrate()` is called, the starting values, bounds, and phases have been reconstructed from the calibration files; the objective function has been created from the wrapper, settings, and observed data; the run directory and executable resources have been prepared; the parallel backend has been configured; and restart behaviour and other controls have been standardised. At this point, the OSMOSE calibration has become a standard `calibrar` optimisation problem. This is precisely the value of the prepared workflow: the complexity of the simulator is absorbed upstream, so that the optimiser still sees the familiar interface of `par`, `fn`, `lower`, `upper`, `phases`, `control`, and `parallel`. For non-OSMOSE readers, this is the main design lesson of the appendix. A complex model does not need a special optimiser API. What it needs is a careful layer of preparation that translates model-specific resources into the generic inputs expected by the optimiser.

The script ends with minimal failure handling and cluster cleanup.

```
if(inherits(opt, "try-error"))
  message("Error while running the calibration.")

if(isTRUE(parallel)) stopCluster(cl)
```

The use of `try()` here is intentionally simple. The script reports that the calibration failed, but leaves the detailed diagnosis to the files produced during setup, testing, and model execution. This is reasonable in a workflow where failures can originate from many places: invalid parameter combinations, simulator crashes, inconsistent files, or platform issues. The cleanup step is more important than it may look. When calibrations are launched repeatedly on shared systems, stale clusters or unclosed worker processes can quickly become a practical problem. Stopping the cluster explicitly keeps the workflow predictable and reduces interference with later jobs.

Although `.calibration.R` is specific to OSMOSE, the overall implementation pattern is generic. A similar launch script for another file-based simulator would typically need the same conceptual blocks: parse execution arguments and scheduler context; read a control file and

prepare a run directory; source a model-specific `run_model()` wrapper; reconstruct the settings table, observed data, starting values, bounds, and phases; build the objective function with `calibration_objFn()`; configure the parallel backend and restart behaviour; and call `calibrate()`. What will change from one model to another is not the overall architecture, but the model-specific details: how parameters are written to disk, how the simulator is launched, how outputs are read and transformed, what additional objects must be exported to the workers, and how internal model parallelism interacts with optimiser-level parallelism. This is why the OSMOSE helper workflow is useful as an example beyond OSMOSE itself. It shows that the main difficulty of calibrating a complex model is usually not the optimisation call itself. The main difficulty is designing a reproducible, scheduler-aware bridge between a model-specific execution environment and the generic optimiser interface.

Seen from that perspective, the purpose of `.calibration.R` is not to implement an optimisation method. That work is delegated to `calibrar`. Its purpose is to assemble a standard optimisation problem from the prepared OSMOSE calibration workspace. It reconstructs the calibration objects from disk, turns the OSMOSE wrapper into an objective function, configures execution control, restart behaviour, and parallel evaluation, and finally passes the result to `calibrate()`. That is the main implementation idea to retain, both for OSMOSE users and for modellers adapting the same pattern to other complex models.

# 14 Appendix: How `run_model.R` turns OSMOSE parameters into calibration outputs

The purpose of `run_model.R` is to implement the model-side bridge between `calibrar` and the OSMOSE core simulator. Each time the optimiser evaluates a candidate parameter set, the objective function calls `run_model()`. The wrapper then reconstructs the OSMOSE parameters implied by that calibration point, writes a runnable parameter file, launches the simulator, reads the resulting outputs back into R, and returns a named list of calibration variables. For OSMOSE users, this appendix provides a more detailed reference for what happens during one model evaluation. For users of other complex simulators, it also illustrates a generic implementation pattern for a black-box `run_model()` wrapper.

The `run_model()` wrapper is defined as a regular R function. Its main argument, `par`, is the candidate calibration parameter set proposed by the optimiser. The argument `conf` provides the prepared OSMOSE calibration configuration, which contains the information needed to interpret and reconstruct model parameters. The argument `osmose` gives the path to the OSMOSE executable used to run the simulator. The argument `is_a_test` indicates whether the wrapper is being called in test mode rather than as part of a full calibration run. The argument `version` specifies the OSMOSE version to use when running and reading the model. The argument `options` passes Java runtime options to the OSMOSE executable. Finally, `...` allows additional arguments to be forwarded if needed. Taken together, these arguments define the information required for the wrapper to evaluate one candidate parameter combination and return the structured list of calibration outputs expected by `calibrar`.

```
run_model = function(par, conf, osmose, is_a_test=FALSE, version="4.3.3",
                     options="-Xmx3g -Xms1g", ...) {
```

The first part of the function establishes the objects that will be needed during the evaluation. It records timing information, extracts basic dimensions from the calibration configuration, and

pulls out the parameter subsets that will later be used to define random-effect-like penalty components.

```
.t0 = Sys.time() # timing
nspp = get_species(conf, type="focal", code=TRUE)
nfsh = get_fisheries(conf, code=TRUE)
ndtperyear = get_par(conf, 'simulation.time.ndtperyear')
nyear      = get_par(conf, 'simulation.time.nyear')
ndt = nyear*ndtperyear

# get deviates to compute random effects
larval_deviates  = get_par(par, 'osmose.user.larval.deviate.log')
fishing_deviates = get_par(par, 'fisheries.rate.byperiod.log')
```

At this stage, the function is not yet interacting with the simulator. It is still working entirely on the calibration-side representation of the problem. This is an important conceptual point. The object `par` received by `run_model()` is not simply a ready-to-run OSMOSE configuration. It is a calibration parameter object that may contain transformed parameters, grouped parameters, helper parameters, or penalty-related components. The wrapper therefore begins by identifying the pieces that will later need to be translated into simulator inputs or returned as auxiliary outputs.

The next block performs the most important parameter reconstruction step: it expands grouped calibration parameters back into model-level OSMOSE parameters, reconstructs the interannual larval mortality trajectories from calibrated deviates, and rebuilds fishery selectivity terms that are represented more compactly during estimation.

```
# recover parameters modelled by groups
random = NULL
nms = get_par(conf, "calibration.*.bygroup$", unlist=TRUE)
if(!is.null(nms)) {
  nms = names(nms)[which(nms)]
  if(length(nms)>0) {
    nms = gsub(nms, pattern="^calibration.", replacement="")
    nms = gsub(nms, pattern=".bygroup$", replacement="")
    for(nm in nms) {
      par = osmose:::get_osmose_parameter(par=par, conf=conf, nm=nm)
      irandom = paste0("random.", nm)
      random = c(random, get_par(par, irandom, as.is=TRUE))
      par = get_par(par, irandom, invert = TRUE, as.is=TRUE)
    }
  }
}

# create interannual variability in larval mortality
for(isp in nspp) {
  nn = sprintf('mortality.additional.larva.rate.seasonality.sp%s', isp)
  ldev = get_par(larval_deviates, sp=as.numeric(isp))
  if(is.null(ldev)) next
```

```
  par[[nn]] = exp(calibrar::spline_par(ldev, n=ndt)$x)
}

# fishing selectivity
d75 = get_par(par, "selectivity.delta75.fsh") # all of them
l50 = get_par(par, "selectivity.l50.fsh") # all of them
L50 = get_par(conf, "selectivity.l50.fsh")

for(ifsh in nfsh) {
  nn = sprintf('fisheries.selectivity.l75.fsh%s', ifsh)
  this.d75 = get_par(d75, fsh=as.numeric(ifsh))
  if(is.null(this.d75)) next
  this.l50 = get_par(l50, fsh=as.numeric(ifsh))
  if(is.null(this.l50)) this.l50 = get_par(L50, fsh=as.numeric(ifsh))
  par[[nn]] = this.l50 + this.d75
}
```

This is where the difference between the calibration parameter layer and the simulator parameter layer becomes explicit. Some parameters are estimated in grouped form because that representation is more stable or more meaningful for optimisation, but OSMOSE ultimately needs species-specific parameters. The helper `get_osmose_parameter()` performs that expansion and also extracts associated random-effect quantities. Likewise, the calibrated larval mortality deviates are not themselves the final time series required by OSMOSE. They are interpolated with `spline_par()` and exponentiated to rebuild the full interannual seasonal trajectory expected by the simulator. Fishery selectivity is treated in a similar way: a more convenient calibration representation is converted back into the final `l75` values required at runtime. For non-OSMOSE users, this block illustrates a very general lesson: a good `run_model()` wrapper often has to reconcile two parameterisations, one designed for estimation and one designed for simulation.

The script then exposes an explicit user-editable block.

```
### BEGIN USER ADDED

### END USER ADDED
```

This block is intentionally empty in the default helper workflow, but it is conceptually important. It acknowledges that some applications will need one last model-specific transformation that cannot be anticipated by the generic helpers. This is the natural place to add extra parameter reconstruction, derived inputs, or one-off adjustments needed by a particular calibration. That design is useful beyond OSMOSE. In many complex-model workflows, a wrapper becomes much more maintainable if it provides one clear extension point rather than requiring users to rewrite the whole launcher.

Once the parameter object has been fully reconstructed, the wrapper converts it into a runnable OSMOSE input file.

```
# remove all osmose.user parameters and clean-up
par = get_par(par, "osmose.user", invert = TRUE, as.is=TRUE)
par = get_par(par, linear=TRUE, as.is = TRUE) # write parameters in linear scale.
```

```
write_osmose(par, file='calibration_parameters.osm')

names(larval_deviates)  = paste("M.larva", get_species(conf, nm=names(larval_deviates)), s
names(fishing_deviates) = paste("F.byperiod", get_fisheries(conf, nm=names(fishing_deviate
```

This block is the boundary between the optimiser-facing and simulator-facing sides of the workflow. Helper-only parameters stored under `osmose.user` are removed because they are useful during reconstruction but should not appear in the final runtime file. The remaining parameters are then converted back to the linear scale expected by OSMOSE and written to `calibration_parameters.osm`. Conceptually, this is the point where one candidate calibration proposal becomes a concrete simulator input. At the same time, the names of the larval and fishing deviates are standardised so that they can later be returned as additional components of the calibration output.

The next stage is the actual execution of OSMOSE and the extraction of the calibration variables.

```
# run osmose!
t0 = t1 = 0 # init times
if(!isTRUE(is_a_test)) {
  t0 = run_osmose(input='osmose-calibration.osm', output='output', osmose=osmose,
              version = version, options=options, verbose=FALSE)$elapsed
}

output = read_osmose(path='output', version=version, null.on.error=TRUE)
cal_output = osmose_calibration_outputs(output)
tr = output$elapsed
```

At this point, the wrapper is no longer transforming parameters; it is running the model. The call to `run_osmose()` launches the OSMOSE core simulator with the prepared calibration configuration and the just-written parameter file. The outputs are then read back into R with `read_osmose()`. Importantly, the wrapper still does not return raw simulator outputs. Instead, it reduces them to the smaller list of calibration-relevant variables with `osmose_calibration_outputs()`. This is another key architectural point. The optimiser does not need the full state of the simulator. It needs only those outputs that will be confronted with observations or used as penalty terms. The wrapper therefore acts as both launcher and reducer: it executes the model and then translates the result into the named calibration-output object expected by the objective function.

The script also includes a simple retry mechanism and a controlled failure path.

```
if(is.null(cal_output)) {
  # sometimes, you get a 'no-write' error. Restarting the calibration usually fix the prob
  # several things can be the issue. Most times, you won't see the error again.
  # Those are caused by the ghosts in the machine (DATARMOR). To please them, we will run
  # model again, and see.
  if(!isTRUE(is_a_test)) {
    t1 = run_osmose(input='osmose-calibration.osm', output='output', osmose=osmose,
```

```
                version = version, options=options, verbose=FALSE)$elapsed
    }

    output = read_osmose(path='output', version=version, null.on.error=TRUE)
    cal_output = osmose_calibration_outputs(output)
    tr = tr + output$elapsed
  }

  # if is still NULL, we will let calibrar to deal with it.
  if(is.null(cal_output)) {
    message("Something wrong happened while running 'run_model'. Returning NULL. Check the '
    return(invisible(cal_output))
  }
```

This is a pragmatic piece of robustness logic. Long-running file-based simulators can fail occasionally for transient reasons, especially on shared or HPC systems. The wrapper therefore retries the OSMOSE run once if no valid calibration outputs are obtained on the first attempt. Only if the second attempt also fails does it return `NULL` and let `calibrar` handle the failed evaluation. This is a useful design pattern for other complex models as well: a wrapper should be robust enough to tolerate occasional execution problems, but should still fail cleanly when an evaluation cannot be recovered.

Finally, the wrapper builds the returned object, records timing information, and exits.

```
  # create final output list, including random effects
  cal_output = c(cal_output, random, random=larval_deviates, random=fishing_deviates)

  # save timing information.
  osmose:::.timing(.t0=.t0, t0=t0, t1=t1, tr=tr, file="../timing.log")

  return(invisible(cal_output))

}
```

The returned object is not only a collection of direct simulator outputs. It also includes the random-effect-like quantities associated with grouped parameters, as well as the larval and fishing deviates. This matters because the objective function may use such terms as penalties or regularisation components alongside direct observational targets. The wrapper therefore returns the full calibration-output object expected by the optimisation workflow, not just the variables that happen to be written by OSMOSE. The timing call serves a different purpose: it records how long the evaluation took, which is useful when calibrations are long and performance needs to be monitored across many model runs.

Seen as a whole, `run_model.R` is not simply a launcher for OSMOSE. It is a translation layer between three different representations of one model evaluation: the calibration parameter set proposed by the optimiser, the runnable parameter file required by the OSMOSE core simulator, and the named calibration outputs expected by the objective function. That is the main implementation idea to retain, both for OSMOSE users and for modellers adapting the same pattern to other complex models. The parameter names, file formats, and helper functions

will differ from one simulator to another, but the architecture remains the same: reconstruct model parameters, write runnable inputs, execute the simulator, read outputs, reduce them to calibration variables, and return a named list.

# 15 Appendix: Example scheduler launch scripts for OSMOSE calibration

The following launch scripts are included here as brief implementation examples. They show how the prepared calibration workspace can be submitted to two popular HPC workload managers: OpenPBS and Slurm. Their role is not to introduce these schedulers in detail, but to illustrate how the same calibration launcher can be wrapped for different execution environments.

In both cases, the underlying logic is the same: move to the submission directory, expose the user R library path, optionally load the required software stack, and then launch `.calibration.R` in OMP mode with an explicit core count and log redirection. The main similarity between the two scripts is therefore conceptual rather than syntactic: both are thin scheduler wrappers around the same calibration launcher. The main differences are due to the scheduler interface itself and their specific directives. Additionally, the Slurm version creates a temporary `PBS_NODEFILE` so that the calibration script can reuse the same node-detection logic as with PBS. Aside from those scheduler-specific details, the scripts are intentionally very similar.

**Example 1. PBS launcher for an OMP calibration run.**
This script illustrates a PBS submission wrapper. The PBS directives define the requested resources, the script changes to the submission directory, sets the user library path, loads the software modules, and finally launches `.calibration.R` with the requested core count.

```
#!/bin/csh
#PBS -N osmose_calibration
#PBS -q omp
#PBS -l select=1:ncpus=56:mem=256g
#PBS -l walltime=240:00:00

source /usr/share/Modules/3.2.10/init/csh

# change to work directory
cd $PBS_O_WORKDIR




# export R_LIBS with the desired version of osmose and calibrar
setenv R_LIBS /home1/datawork/$USER/libs/R/lib

# Load modules (if available, choose versions working in parallel)
module load R/3.6.3-intel-cc-17.0.2.174
module load java/openjdk-16.0.2
module load nco
```

```
# Load conda environment (if available)
#conda activate r4

# Run R script in parallel mode
time Rscript .calibration.R --ncores=56 --omp --nht >>& mycalibration.log
```

**Example 2. Slurm launcher for an OMP calibration run.**
This script illustrates the same workflow under Slurm. Its structure closely mirrors the PBS version, but uses Slurm directives and creates a `PBS_NODEFILE` from `srun hostname` so that the calibration launcher can keep the same environment-detection logic.

```
#!/bin/bash
#SBATCH --job-name=osmose_calibration
#SBATCH --partition=standard
#SBATCH -c 61
#SBATCH --mem=64G

# change to work directory
cd $SLURM_SUBMIT_DIR

export PBS_NODEFILE=$(mktemp)  # Create a temporary file for the node list
scontrol show hostnames $SLURM_JOB_NODELIST > "$PBS_NODEFILE"

# export R_LIBS with the desired version of osmose and calibrar
export R_LIBS=/home/$USER/R/x86_64-pc-linux-gnu-library/4.4

# Load modules (if available, choose versions working in parallel)
#module load R
#module load java
#module load nco

# Load conda environment (if available)
#conda activate r4

# Run R script in parallel mode
time Rscript .calibration.R --ncores=61 --omp --nht &>> mycalibration.log
```

These examples are meant for illustration only. In practice, users will usually need to adapt queue or partition names, memory and wall-time requests, module names, library paths, and possibly the number of requested cores to match their own platform. The important point is that the scheduler script should remain a lightweight execution wrapper, while the calibration logic itself stays inside the prepared workspace and the `.calibration.R` launcher.

Oliveros-Ramos, Ricardo, Philippe Verley, Vincent Echevin, and Yunne-Jai Shin. 2017. 'A Sequential Approach to Calibrate Ecosystem Models with Multiple Time Series Data'. *Progress in Oceanography* 151: 227–44. https://doi.org/10.1016/j.pocean.2017.01.002.